



TEXT SEARCH ALGORITHMS AND MULTIMEDIA INFORMATION RETRIEVAL

UNIT-V

Text Search Algorithms: Introduction to Text Search Techniques, Software Text Search Algorithms, Hardware Text Search Systems

Multimedia Information Retrieval: Spoken Language Audio Retrieval, Non-Speech Audio Retrieval, Graph Retrieval, Imagery Retrieval, Video Retrieval

Text Search Techniques

Three classical methods for organizing and searching textual databases are mentioned:

- **Full Text Scanning (Streaming):** Directly scanning the entire text to find matches for a query.
- **Word Inversion (Indexing):** Creating an index of words to quickly locate relevant items, reducing the data to be searched.
- **Multiattribute Retrieval:** Searching based on multiple attributes of the data (e.g., metadata and content).

- The diagram shows the architecture of a text streaming search system, which includes three main components:
- **Database:** Stores the full text of items to be searched.
- **Term Detector:** A hardware/software module that identifies search terms in the text stream. It checks for the presence of query terms and may also handle basic logic (e.g., AND/OR relationships between terms).
- **User Interface:** Displays search results and status to the user, allowing retrieval of matching items.

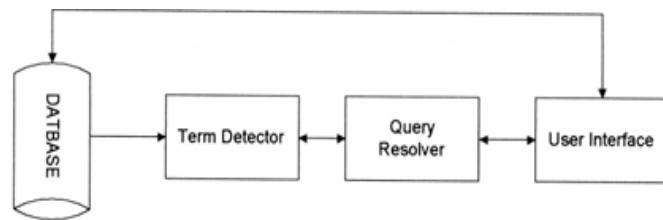


Figure 9.1 Text Streaming Architecture

- **Query Resolver:**
- Takes user queries, extracts search terms and logic, and sends the terms to the Term Detector.
- Receives results from the Term Detector, evaluates the query logic, and determines which items satisfy the query (possibly assigning weights to matches).

How it works:

1. The database streams text to the Term Detector.
2. The Term Detector identifies query terms in the stream and sends detected terms to the Query Resolver.
3. The Query Resolver processes the results, applies query logic, and passes the final results to the User Interface for display.
4. This system allows results to be shown as soon as a match is found, unlike indexing, which typically processes the entire query first.

Finite State Automata (FSA) in Text Search

- Text search systems often use finite state automata (FSA) to efficiently detect patterns in a text stream.
- An FSA is a logical machine defined by five elements (as per the text):
- Automata Definition:** This defines an FSA to detect the string "CPU" in a text stream.
- The components are:**

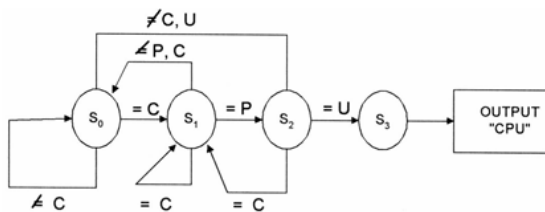


Figure 9.2a Finite State Automata

I = set of all alphabetic characters
 S = set $\{S_0, S_1, S_2, S_3\}$
 P = set $\{S_0 \rightarrow S_1 \text{ if } I = C\}$
 $S_0 \rightarrow S_0 \text{ if } I \neq C$
 $S_1 \rightarrow S_2 \text{ if } I = P$
 $S_1 \rightarrow S_0 \text{ if } I \neq \{P, C\}$
 $S_1 \rightarrow S_1 \text{ if } I = C$
 $S_2 \rightarrow S_3 \text{ if } I = U$
 $S_2 \rightarrow S_1 \text{ if } I = C$
 $S_2 \rightarrow S_0 \text{ if } I \neq \{C, U\}$

$S_0 = \{S_0\}$
 $S_F = \{S_3\}$

Figure 9.2b Automata Definition

I - a set of input symbols from the alphabet supported by the automata
 S - a set of possible states
 P - a set of productions that define the next state based upon the current state and input symbol
 S_0 - a special state called the initial state
 S_F - a set of one or more final states from the set S

How it Works:

The diagram is a directed graph where:

- Nodes represent states (S_0, S_1, S_2, S_3).
- Edges represent transitions based on input symbols ('C', 'P', 'U', or others like $\bar{C}$ for "not C").

- Start at S_0 (initial state).
- If the input is 'C', move to S_1 . Otherwise, stay in S_0 .
- In S_1 :
 - If the next input is 'P', move to S_2 .
 - If the input is 'C', stay in S_1 .
 - Otherwise, return to S_0 .

- In S_2 :
 - If the next input is 'U', move to S_3 (final state, indicating "CPU" was found).
 - If the input is 'C', move back to S_1 (to handle cases like "CCPU").
 - Otherwise, return to S_0 .
- Reaching S_3 (final state) means the pattern "CPU" was found in the stream.

Output:

- When the FSA reaches S_3 , it outputs "CPU" to indicate a match.



Software Text Search Algorithms

1. Brute Force
2. Knuth-Morris-Pratt (KMP)
3. Boyer-Moore
4. Aho-Corasick
5. Shift-Add

1. Brute Force

- The **Brute Force method is the simplest string-matching** algorithm.
- It compares the search pattern with the input text **character by character, starting from the first position**.
- If a **mismatch** occurs, the pattern **shifts one position to the right**, and the comparison restarts.
- The expected number of comparisons for an input text of length n and a pattern of length m with an alphabet size c is given by the formula in the image:

$$N_c = \frac{c}{c-1} \left(1 - \frac{1}{c^m}\right) (n - m + 1) + O(1)$$

Explanation of the Formula: This formula calculates the expected comparisons based on the probability of matches/mismatches over an alphabet of size c . The term $\frac{c}{c-1}$ accounts for the likelihood of a character mismatch, and $1 - \frac{1}{c^m}$ adjusts for the probability of the pattern not matching randomly. The $(n - m + 1)$ represents the number of possible starting positions in the text, and $O(1)$ is a constant for edge cases.

Efficiency:

- **Worst Case:** $O(n \cdot m)$, as it may need to compare the entire pattern at every position.
- **Preprocessing:** $O(1)$, since no preprocessing is required.

Example

- The document doesn't provide a specific example for Brute Force, but let's consider the input stream and search pattern from the KMP example:
- Input Stream: **a, b, d, a, d, e, f, g**
- Search Pattern: **a, b, d, f**
- Start at position 1: **a** matches **a**, **b** matches **b**, **d** matches **d**, but **f** mismatches with **a**. Shift one position.
- Position 2: **b** mismatches **a**. Shift again.
- This continues, checking each position, leading to many comparisons.

2. Knuth-Morris-Pratt (KMP)

- The KMP algorithm improves on Brute Force by avoiding unnecessary comparisons.
- When a mismatch occurs, **KMP uses information from previously matched characters** to determine how far to shift the pattern, **rather than shifting one position at a time**.
- This is achieved by preprocessing the pattern to create a **Shift Table (also called a failure function or prefix function)**, which indicates **how many positions to skip based on the mismatch**.

Example:

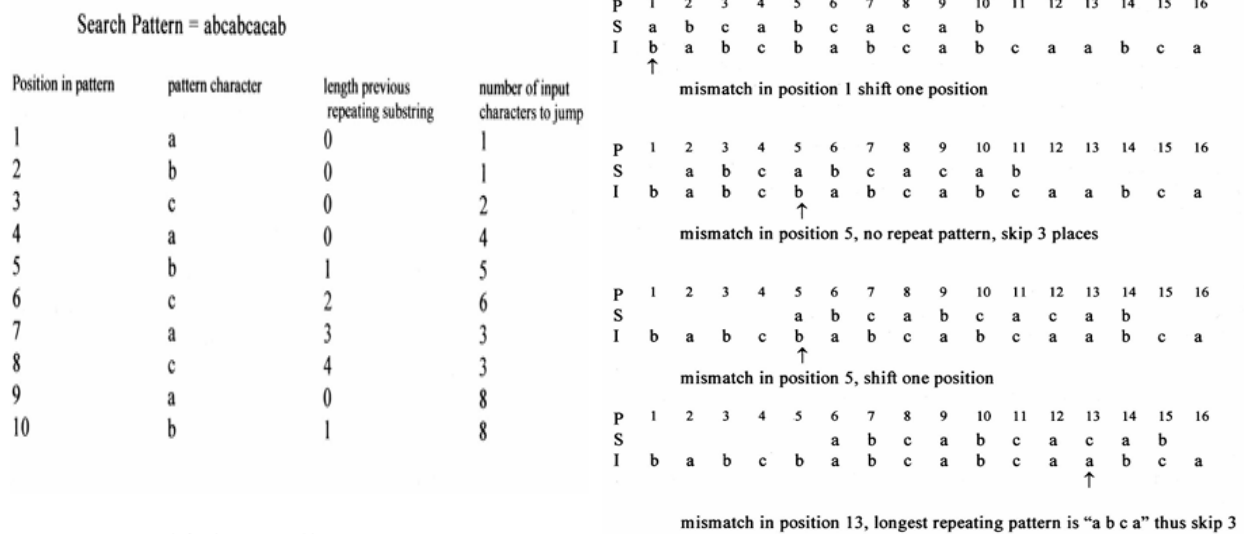


Figure 9.3 Shift Characters Table

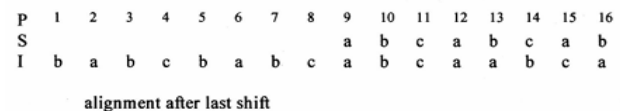


Figure 9.4 Example of Knuth-Morris-Pratt Algorithm

Steps:

1. **First Comparison** (Position 1):

- $S : a$ mismatches $I : b$. Shift 1 position (from Shift Table).
- New alignment: S starts at position 2.

2. **Position 5** (Mismatch):

- $S : ab$ matches $I : ab$, but $S : c$ mismatches $I : b$.
- Shift Table at position 3 (c) says jump 4 positions (no repeat of pattern, skip 3 places).
- New alignment: S starts at position 8.

3. **Position 5 Again** (Mismatch):

- After shifting, $S : ab$ matches, but $S : c$ mismatches $I : b$.
- Shift 1 position (to align repeating "ab").

4. Position 13 (Mismatch):

- Longest repeating pattern is *abac*, so skip 3 positions to align the next occurrence.

5. Final Alignment:

- After the last shift, the pattern aligns perfectly at position 13.

Efficiency:

- Preprocessing:** $O(m)$, to build the Shift Table.
- Search:** $O(n)$, linear time, as it avoids backtracking.
- Total:** $O(n + m)$.

3. Boyer-Moore

- Boyer-Moore enhances efficiency by comparing the pattern from right to left (**unlike Brute Force and KMP, which go left to right**).
- When a mismatch occurs, it uses two rules to determine the shift:
 - Bad Character Rule:** Shift the pattern to align the mismatched character in the input stream with its next occurrence in the pattern (or skip the pattern entirely if the character doesn't exist in the pattern).
 - Good Suffix Rule:** If a **suffix of the pattern matches**, shift to align the next occurrence of that suffix in the pattern.

Compare from right: *b* matches *b*, *a* matches *a*, *a* matches *a*, but *d* mismatches *f*.

Bad Character Rule: *f* doesn't exist in the pattern, so skip 3 positions (the length of the matched suffix *aab*).

Good Suffix Rule: The matched suffix *aab* repeats at the start of the pattern (*ab*). Shift 4 positions to align *ab* over positions 6 and 7.

Shift: Take the maximum (4 positions). New alignment starts at position 5.

the search pattern is (*a b d a a b*) and the alphabet is (*a, b, c, d, e, f*) with $m = 6$ and $c = 6$.

Position	1	2	3	4	5	6	7	8	9	10	11	12	13
Input Stream	f	a	b	f	a	a	b	b	d	a	b	a	b
Search Pattern		a	b	d	a	a	b						
				↑									

a. mismatch in position 4: $ALGO_1 = 3$, $ALGO_2 = 4$, thus skip 4 places

After shifting: *b* matches *b*, but *a* mismatches *d*.

Bad Character Rule: *d* exists in the pattern at position 4. Shift 1 position to align *d*.

Good Suffix Rule: No significant suffix match to consider.

Shift: 1 position. New alignment starts at position 10.

Position	1	2	3	4	5	6	7	8	9	10	11	12	13
Input Stream	f	a	b	f	a	a	b	b	d	a	b	a	b
Search Pattern						a	b	d	a	a	b		

b. mismatch in position 9: $ALGO_1 = 1$, $ALGO_2 = 4$ thus skip four places

Position	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
Input Stream	f	a	b	f	a	a	b	b	d	a	b	d	a	a	b
Search Pattern										a	b	d	a	a	b

c. new aligned search continues with a match

Figure 9.5 Boyer-Moore Algorithm

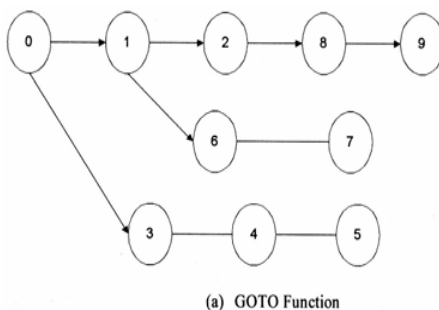
After the next shift of 4 positions, the pattern matches perfectly at positions 10-15.

Efficiency:

- **Preprocessing:** $O(m)$, to build the bad character and good suffix tables.
- **Search:** At most $O(n + m)$, though often sublinear in practice due to large jumps.
- **Best Case:** $O(n/m)$, when large skips are possible.

4. Aho-Corasick

- The Aho-Corasick algorithm is designed for **multiple pattern matching**.
- It uses a **finite state machine (FSM)** to process the input text and match multiple patterns in one pass.
- The FSM is defined by three functions:
 - **GOTO Function:** A state transition graph (Figure 9.6a) that defines the next state based on the current state and input character.
 - **Failure Function:** When a mismatch occurs, this function (Figure 9.6b) specifies the state to fall back to.
 - **Output Function:** Specifies which patterns are matched at each state (Figure 9.6c).



State 0: Input $H \rightarrow$ State 1, others $\rightarrow$ State 0.

State 1: Input $E \rightarrow$ State 2 (matches HE), $I \rightarrow$ State 6, $S \rightarrow$ State 4.

i	1	2	3	4	5	6	7	8	9
f(i)	0	0	0	1	2	0	3	0	3

(b) Failure Function

$f(1) = 0$: If at state 1 and mismatch, go back to state 0.

$f(4) = 1$: If at state 4 (after S) and mismatch, go to state 1.

state	2	5	7	9
OUTPUT	HE	SHE, HE	HIS	HERS

(c) OUTPUT Function

State 2: Outputs HE .

State 5: Outputs SHE, HE .

State 7: Outputs HIS .

State 9: Outputs $HERS$.

Efficiency:

- **Preprocessing:** $O(m)$, where m is the total length of all patterns.
- **Search:** $O(n)$, linear in the input text length.
- The number of state transitions is independent of the number of patterns, a major improvement over KMP for multiple patterns.

5. Shift-Add

- The Shift-Add algorithm, developed by Baeza-Yates and Gonnet, **handles fuzzy matching (allowing mismatches, "don't care" symbols, and complements)**.
- It uses a vector $s(j)$ to track the number of mismatches between the pattern and the text at position j . The vector is updated as the text is processed:

$$s(j) = s(j-1) + T(\text{text}(j))$$

- $T(x) = 0$ if x matches the pattern at the corresponding position, otherwise $T(x) = 1$.
- If $s(j) = 0$, a perfect match is found. Otherwise, $s(j)$ indicates the number of mismatches.
- The search pattern be **ababc (m = 5)**
- A segment of input text to be **cbbabababcaba**
Compare *ababc* with *cbbab*:

Vector Position 1: a vs c , mismatch $\rightarrow 1$.

Vector Position 2: ab vs cb , 2 mismatches $\rightarrow 2$.

Vector Position 3: aba vs cbb , 3 mismatches $\rightarrow 3$.

Vector Position 4: $abab$ vs $cbba$, 3 mismatches $\rightarrow 0$ (matches at b).

Vector Position 5: $ababc$ vs $cbbab$, 4 mismatches $\rightarrow 1$.

Vector: $[1, 2, 3, 0, 1]$, so $T(old) = [1, 0, 1, 0, 1]$.

	Vector value	Vector Position
a b a b c	1	1
a b a b c	0	2
a b a b c	3	3
a b a b c	0	4
a b a b c		5
... c b b a b a b a b c a b a ...	Input Text Stream ...	

Figure 9.7 Vector for Position $j - 1$

Shift one position: Compare $ababc$ with $bbaba$

Vector Position 1: a vs b , mismatch $\rightarrow 0$.

Vector Position 2: ab vs bb , 1 mismatch $\rightarrow 2$.

Vector Position 3: aba vs bba , 1 mismatch $\rightarrow 0$.

Vector Position 4: $abab$ vs $bbab$, 1 mismatch $\rightarrow 4$.

Vector Position 5: $ababc$ vs $bbaba$, 2 mismatches $\rightarrow 1$.

	Vector value	Vector Position
a b a b c	0	1
a b a b c	2	2
a b a b c	0	3
a b a b c	4	4
a b a b c		5
c b b a b a b a b c a b a ...	Input Text Stream ...	

Figure 9.8 Vector for Position j

Vector: $[0, 2, 0, 4, 1]$, so $T(new) = [0, 1, 0, 1, 1]$

$$s(1, new) = s(0, old) + T_1(new) = 0 + 0 = 0$$

$$s(2, new) = s(1, old) + T_2(new) = 1 + 1 = 2$$

$$s(3, new) = s(2, old) + T_3(new) = 0 + 0 = 0$$

$$s(4, new) = s(3, old) + T_4(new) = 3 + 1 = 4$$

$$s(5, new) = s(4, old) + T_5(new) = 0 + 1 = 1$$

Efficiency:

- **Preprocessing:** Depends on the pattern and alphabet size.
- **Search:** Linear in the text length, with the ability to handle fuzzy matching efficiently.

Extended Features:

- The Shift-Add algorithm can handle:
 - **Don't Care Symbols (*):** Matches any character.
 - **Complements:** Matches characters not in a specified class.
 - **Classes of Characters:** Matches any character in a set.
- Example Pattern: $a\hat{b} * [abc]$, where $\hat{b}$ is "not b ", $*$ is any character, and $[abc]$ matches a , b , or c .

Hardware Text Search Systems

- Hardware text search systems were developed to **offload** the computationally intensive task of searching large text databases from general-purpose computers to specialized hardware.
- This approach emerged in the 1970s due to limitations in software-based search systems, such as:
 - **Scalability Issues:** Software systems struggled to handle many search terms simultaneously against the same text.
 - **I/O Bottlenecks:** The speed of transferring data from secondary storage (like disk drives) to the processor limited search performance.
 - **Indexing Overhead:** Traditional software search systems rely on indexes, which can be 70% the size of the actual data, requiring significant storage and time to build or update.

Hardware Text Search Systems – issues

- Hardware text search systems **address these issues** by:
 - Using **dedicated hardware** to perform searches, freeing up the main computer for user interface and result retrieval.
 - **Eliminating the need for an index**, allowing new data to be searched immediately upon receipt.
 - Providing **deterministic search times**, as the speed depends only on the time to stream data from storage, not on the **complexity of the query or database size**.
- The **maximum search time** in such systems is the time to search one disk, achieved by assigning one hardware search unit per disk. This makes the system highly scalable—adding more **hardware units allows larger databases** to be searched without increasing search time.

Hardware Text Search Systems - Architecture

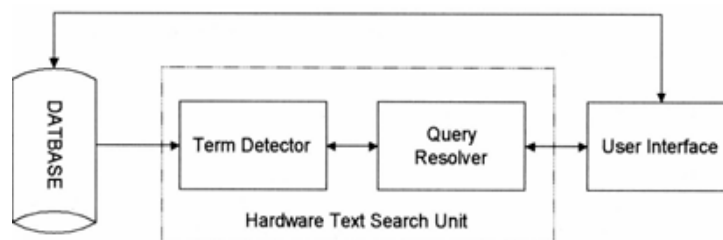


Figure 9.9 Hardware Text Search Unit

Database: Stores the text data on secondary storage (e.g., disk drives).

Term Detector: A hardware component that identifies matches between the query terms and the text stream.

Query Resolver: Processes the matches (e.g., applying Boolean logic) and determines if the text satisfies the query.

User Interface: The main computer that interacts with the user, receiving queries and displaying results.

1. The user submits a query via the user interface.



TEXT SEARCH ALGORITHMS AND MULTIMEDIA INFORMATION RETRIEVAL

2. The query is sent to the hardware text search unit.
3. The term detector scans the text stream from the database, identifying matches.
4. The query resolver evaluates the matches against the query conditions (e.g., Boolean logic, proximity).
5. Results (hits) are sent back to the user interface for display.

Hardware Text Search Systems - Term Detector Implementations

- The term detector is the core of the hardware text search unit, responsible for matching query terms against the text stream.
- Three main approaches to implementing term detectors are mentioned:
 - Parallel Comparators or Associative Memory
 - Cellular Structure
 - Universal Finite State Automata (FSA)

Term Detector - Parallel Comparators or Associative Memory

- **How It Works:**
 - Each query term is **assigned** to a separate comparison element (comparator).
 - The text stream is serially fed into the detector, and each comparator checks for its assigned term in parallel.
 - When a **match is found**, the comparator sets a **status flag**, which is sent to the query resolver (typically on the main computer).
- **Example:**
 - In the **GESCAN system**, some Boolean logic (e.g., AND, OR between terms) is resolved directly in the term detector hardware, reducing the load on the main computer.

Term Detector - Cellular Structure

- This approach uses a series of interconnected cells, each responsible for matching a single character or small part of a query term.
- Cells are typically implemented on LSI (Large-Scale Integration) chips, which can be chained together to handle longer query terms.
- **Example:**
 - In the GESCAN Text Array Processor (TAP), a string of character cells on an LSI chip matches query terms, while a separate resolution chip handles Boolean logic and proximity requirements.

Term Detector - Universal Finite State Automata (FSA)

- The system uses a **finite state machine (FSM)** to represent the query terms as states and transitions.
- The text stream is processed by the FSM, which transitions between states as characters are matched.

- **Example:** The High Speed Text Search (HSTS) machine by Operating Systems Inc. (OSI) uses an algorithm similar to the Aho-Corasick software FSM but runs **three parallel state machines**:
 - i. One for contiguous word phrases.
 - ii. One for embedded term matches.
 - iii. One for exact word matches.
- **Advantages:**
 - Efficient for complex pattern matching, including phrases and partial matches.
 - Can handle multiple queries simultaneously by running parallel FSMs.

Historical Development of Hardware Text Search Systems

- Rapid Search Machine (General Electric, 1970s)
- **Associative File Processor (AFP) by Operating Systems Inc. (OSI)**
- **High Speed Text Search (HSTS) by OSI**
- **GESCAN (General Electric)**
- Fast Data Finder (FDF) by TRW (Later Paracel)
- Other Systems

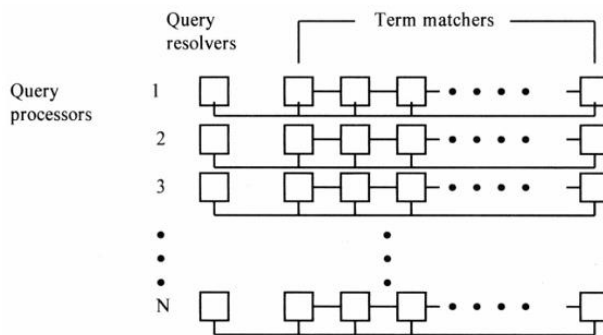


Figure 9.10 GESCAN Text Array Processor

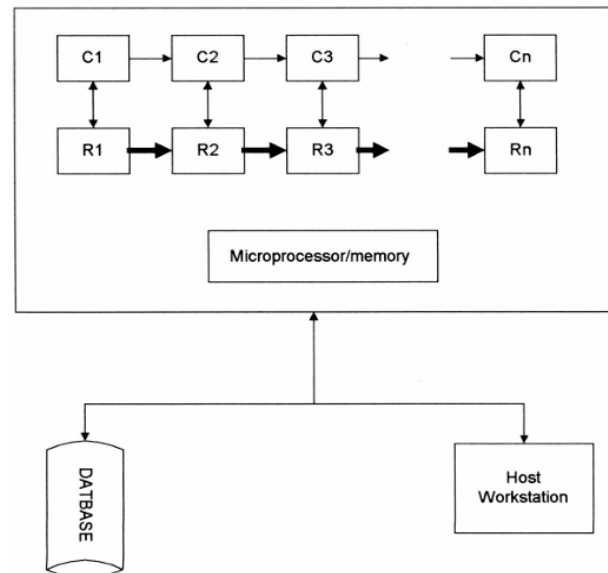


Figure 9.11 Fast Data Finder Architecture

Advantages of Hardware Text Search Systems

- **No Indexing Required:**
 - Eliminates the need for an index (which can be 70% the size of the data), saving storage and allowing immediate searching of new data.
- **Deterministic Search Times:**
 - Search speed depends only on the time to stream data from the disk, making performance predictable.



TEXT SEARCH ALGORITHMS AND MULTIMEDIA INFORMATION RETRIEVAL

- **Real-Time Results:**
 - Hits are delivered to the user as they are found, rather than waiting for the entire search to complete.
- **Scalability:**
 - Adding more hardware units (one per disk) allows the system to handle larger databases without increasing search time.

Modern Applications

- Genetic Analysis
 - The Fast Data Finder (FDF) has found a new application in genetic analysis, as marketed by Paracel:
- Sequence Homology:
 - The FDF is used to compare genetic sequences (e.g., DNA or proteins) to known families, helping identify the functions of newly sequenced genes.
- Fuzzy Matching:
 - The FDF's fuzzy matching capability is particularly useful for genetic sequences, where approximate matches are often more relevant than exact matches.
- Algorithms:
 - Smith-Waterman Algorithm:
 - Used for finding local sequence similarities, optimal for genetic analysis.
 - General Profile Algorithm:
 - Searches for conserved regions in nucleic acids or proteins across evolutionary changes.
 - Biology Tool Kit (BTK):
 - Paracel combines the FDF with BTK software to perform advanced genetic analysis.
 - The FDF identifies sequences with high similarity to the query, and the BTK software completes the analysis (e.g., scoring, alignment).

Multimedia Information Retrieval

Definition: *Multimedia information retrieval is the process of satisfying a user's stated information need by identifying all relevant text, graphics, audio (speech and non-speech audio), imagery, or video documents or portions of documents from a document collection.*

Spoken Language Audio Retrieval

Left Column: Lists speakers (e.g., Elizabeth Vargas, William Cohen).

Center Column: Shows transcribed speech with highlighted named entities (people, organizations, locations).

Right Column: Indicates topics (e.g., "Foreign relations with the United Nations").

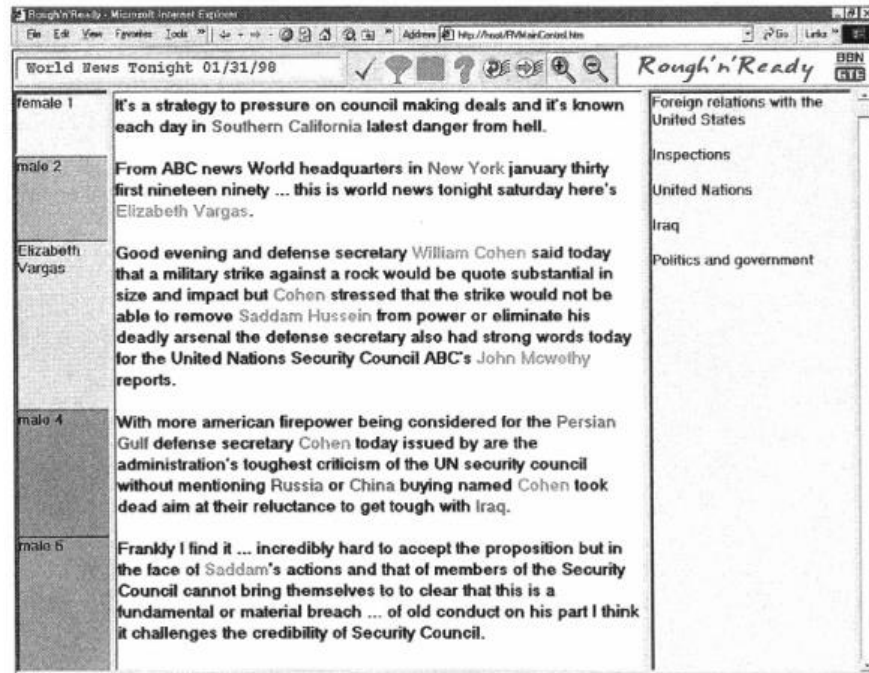


Figure 10.1. BBN's Rough and Ready

Non-Speech Audio Retrieval

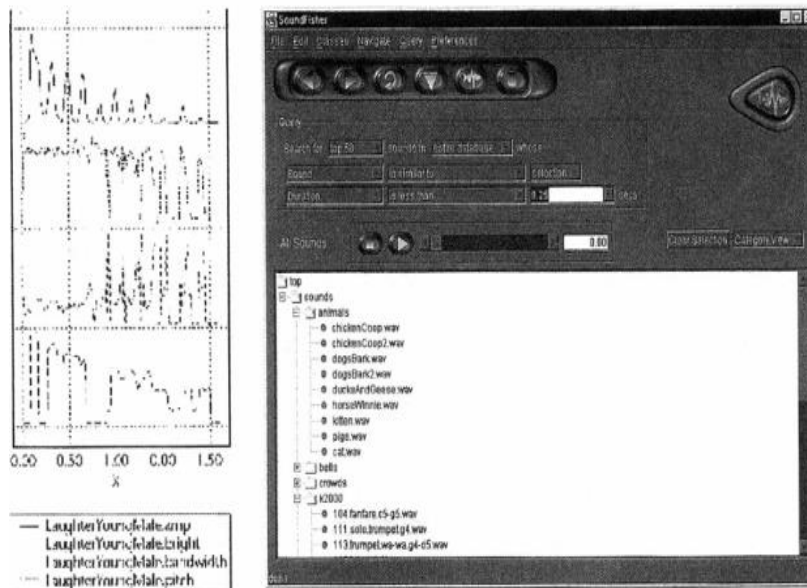


Figure 10.2a. Analysis of Male Laughter. Figure 10.2b. Content based access to audio.



TEXT SEARCH ALGORITHMS AND MULTIMEDIA INFORMATION RETRIEVAL

- Non-speech audio retrieval, such as noises or sounds, which is crucial for fields like music and video production.
- It focuses on the soundfisher system (developed by thorn blum et al., 1997, at www.Musclefish.Com), a user-extensible tool for sound classification and retrieval.
- Non-speech audio retrieval, such as noises or sounds, which is crucial for fields like music and video production.
- It focuses on the soundfisher system (developed by thorn blum et al., 1997, at www.Musclefish.Com), a user-extensible tool for sound classification and retrieval.
- **SoundFisher System Overview**
 - SoundFisher uses techniques from signal processing, psychoacoustics, speech recognition, computer music, and multimedia databases to index and retrieve sounds. Instead of words (as in text retrieval), it indexes sounds using a vector of acoustic features:
 - Directly measurable: duration, loudness, pitch, brightness.
 - These features allow users to search for sounds within specific ranges.
- **Analysis of Male Laughter**
 - a graphical analysis of a male laughter sound across multiple dimensions:
 - **Amplitude:** Loudness over time.
 - **Brightness:** The perceived "sharpness" or "clarity" of the sound.
 - **Bandwidth:** The range of frequencies present.
 - **Pitch:** The perceived frequency of the sound. The graphs (labeled "LAUGHTER_MALE_BRIL") visualize how these features vary, helping to characterize the sound for indexing.
- This figure shows SoundFisher's user interface for browsing and querying a sound database. Features include:
 - A list of sound categories (e.g., animals like "chickenCoop.wav," "dogsBark2.wav").
 - Options to filter by acoustic properties (e.g., pitch, duration) or perceptual properties (e.g., "scratchy").
 - Support for complex queries like finding AIFF-encoded animal or human vocal sounds similar to barking, ignoring duration or amplitude.

Non-Speech Audio Retrieval - System Capabilities and Evaluation

- SoundFisher was tested on a database of 400 diverse sound files (e.g., nature, animals, instruments, speech).
- It supports advanced features like training the system to recognize indirect perceptual properties (e.g., "buzziness").
- Identified needs for improvement include:
 - Better sound displays.
 - Sound synthesis for query refinement.
 - Sound separation.
 - Matching feature trajectories over time.

Graph Retrieval

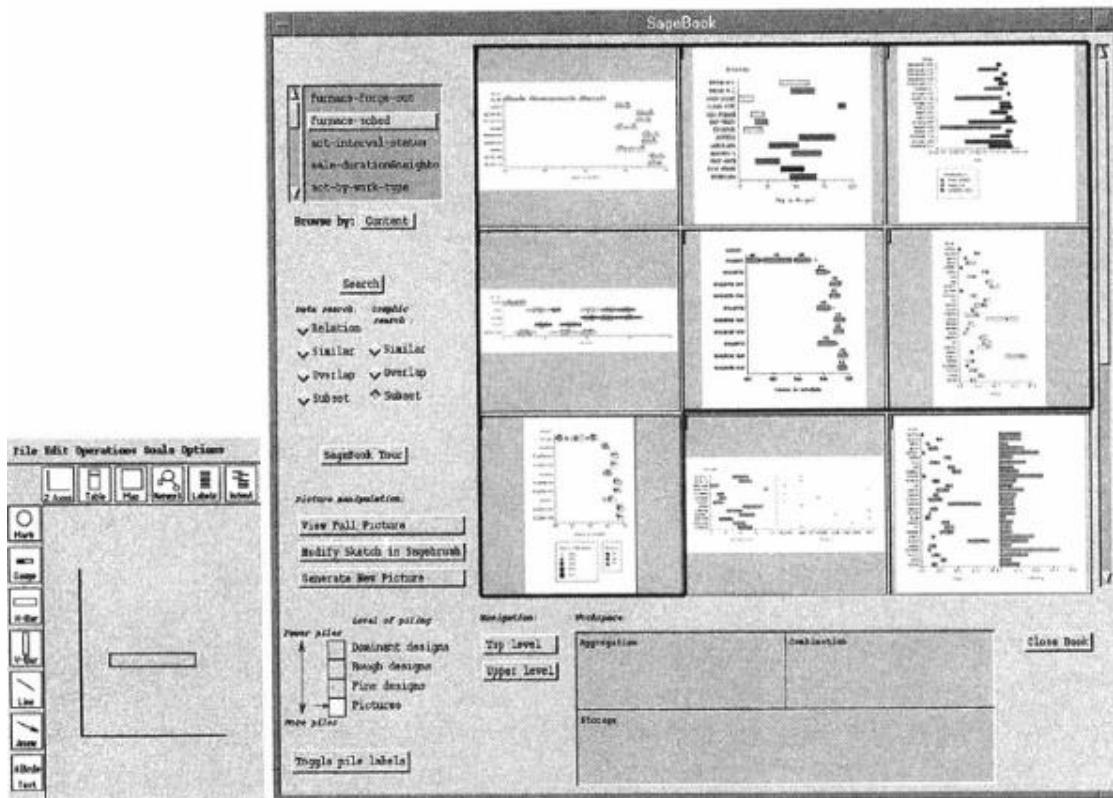


Figure 10.3. SageBrush Query Interface and SageBook display of retrieved relevant graphics

- SageBook is a system for retrieving and adapting data graphics (e.g., bar charts, line graphs, scatter plots) based on their content and properties.
- It supports querying, indexing, and **modifying graphics, similar to how text or audio retrieval systems work but tailored for visual data representations.**
- **Left Side:** The SageBrush interface shows a query being built for a chart with horizontal interval bars.
- **Right Side:** The retrieved graphics are charts with similar properties (one space, horizontal interval bars), highlighted to show they match the query using a “close graphics matching strategy.”

Graph Retrieval - How Does It Work?

1. Query Interface (SageBrush):

- The left side of Figure 10.3 shows the SageBrush interface, where users create a query using a graphical drag-and-drop method.
- Users select and arrange elements like spaces (e.g., a chart area), objects (e.g., bars, lines, marks), and their properties (e.g., color, size, shape).



TEXT SEARCH ALGORITHMS AND MULTIMEDIA INFORMATION RETRIEVAL

- For example, you can specify a chart with horizontal interval bars in a certain color.
2. **Search and Matching:**
- SageBook searches a library of stored graphics by matching the query's graphical elements (called graphemes) and their properties (e.g., bars must match in type, color, shape, etc.).
 - It also matches the underlying data represented by the graphic (e.g., numerical values or categories).
 - The system uses both exact matching (identical properties) and similarity-based matching (close but not identical).
 - In Figure 10.3, the query specifies a chart with one space and horizontal interval bars. The right side of the figure shows retrieved graphics that match this criteria, ranked by similarity.
3. **Adaptation:**
- Retrieved graphics can be modified. SageBook understands the syntax and semantics of graphics, including spatial relationships, data domains (e.g., 2D coordinates), and attributes.
 - Users can manually adapt graphics, or SageBook can automatically adjust them (e.g., removing unmatched elements).
4. **Search Strategies:**
- SageBook offers multiple search strategies (three for graphical properties, four for data properties) with varying levels of strictness.
 - It also supports clustering techniques to organize large collections of graphics for easier browsing.

Graph Retrieval – Applications

- Beyond business graphics, SageBook's approach can be applied to fields like:
 - **Cartography:** Maps with terrain or elevation data.
 - **Architecture:** Blueprints.
 - **Networking:** Diagrams of routers and links.
 - **Military Planning:** Maps with forces and defenses.

Imagery Retrieval

Imagery Retrieval - content-based image retrieval (CBIR) systems like IBM's Query By Image Content (QBIC) system.

- Context and Importance
 - With the rise of digital imagery (e.g., web images, personal photo collections), there's a growing need for efficient image search and retrieval.
 - Traditional methods rely on metadata (like captions or tags), but these are limited because they require manual annotation, which is time-consuming.
 - CBIR systems aim to search images based on their visual content—features like color, texture, shape, or sketches—without needing manual tags.

QBIC System Overview

- The **QBIC** system, developed by Flicker et al. (1997), is a pioneering example of CBIR. It allows users to query image databases using visual properties instead of keywords. For instance:
- **Query by Color:** A user can search for images dominated by a specific color, like red.
- **Query by Shape/Texture:** Users can draw a shape or select a texture to find similar images.
- **Combined Queries:** Users can refine searches by adding keywords to visual queries (e.g., searching for "red stamps" and then narrowing it to "presidents").

This shows the QBIC interface where a user queries a database of U.S. stamps (pre-1995) for those that are predominantly red.

The interface includes options to specify color percentages and adjust the query.

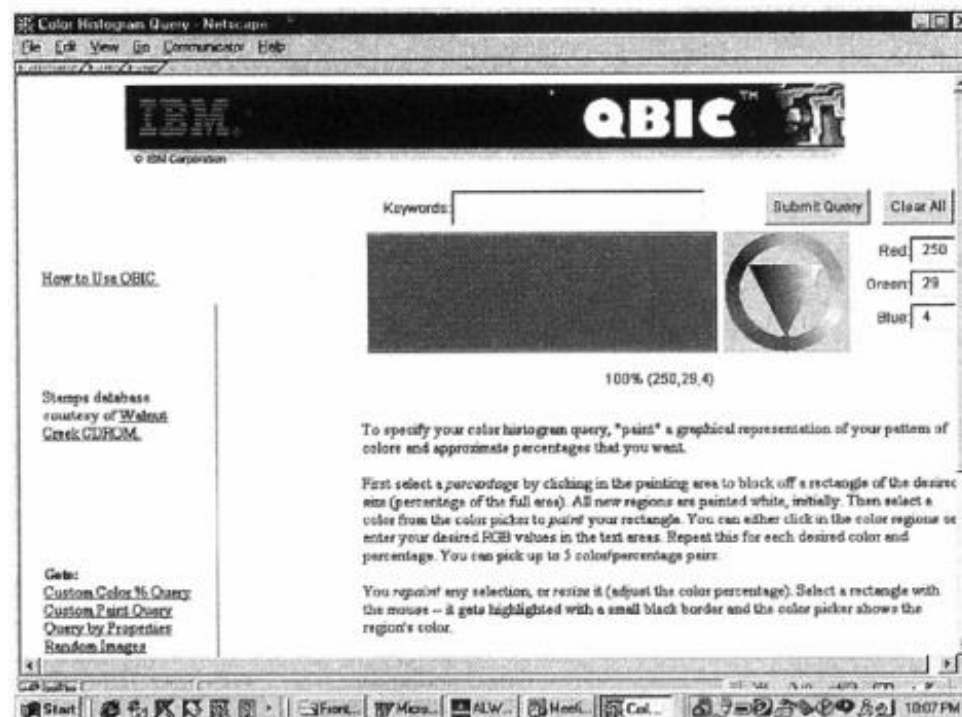


Figure 10.4a. QBIC Query by Color red

This displays the results of the red stamp query—a grid of stamps that are primarily red.

This refines the previous query by adding the keyword "president."

The results show red stamps featuring U.S. presidents, with one exception: the bottom-right stamp is of Martha Washington, part of the presidential stamp collection.

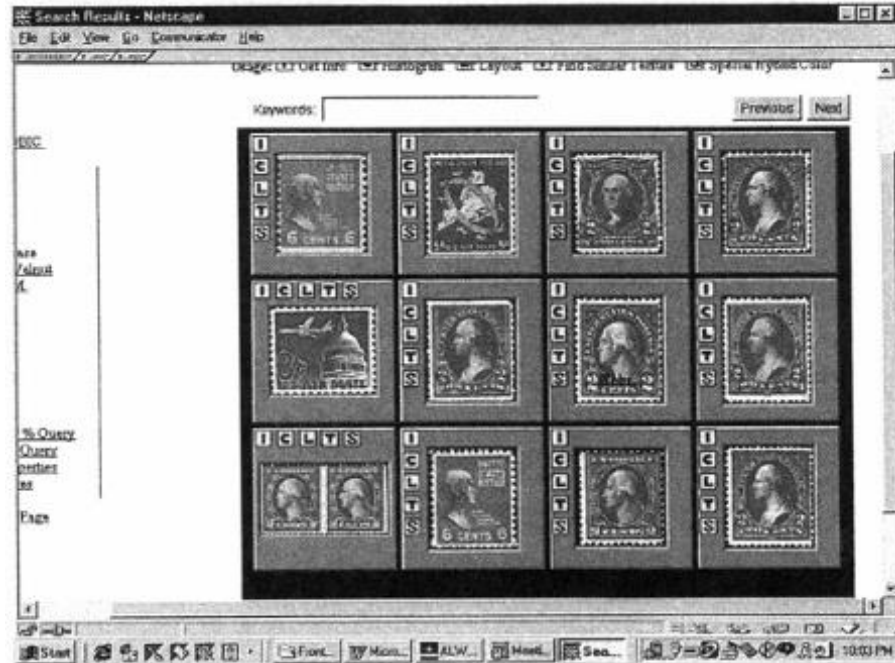


Figure 10.4b. Retrieved red stamps

Advanced Features of QBIC

- QBIC supports complex queries, such as finding images with specific combinations of visual elements (e.g., "a coarsely textured, red round object and a green square"). To make this possible:
- QBIC uses **automated and semi-automated** tools to extract objects from images (e.g., distinguishing foreground from background).
- If text captions are available, they can be used to enhance the search (as seen in Figure 10.4c).

Extending CBIR to video retrieval

- **Shot Detection:** Videos are broken into shots, and a representative frame (called an r-frame or keyframe) is extracted for each shot.
- **Motion Analysis:** QBIC can analyze motion in videos, enabling queries like "find shots panning left to right." Results are shown as ranked r-frames, which act as thumbnails; clicking one plays the associated video shot.

Specialized Applications

- The text highlights other areas of content-based retrieval:

1. Face Processing:

- Face Detection: Identifying faces in an image.



TEXT SEARCH ALGORITHMS AND MULTIMEDIA INFORMATION RETRIEVAL

- Face Recognition: Verifying a face's identity (e.g., the U.S. Immigration Service uses Facelt® at the Otay Mesa border to match drivers' faces with registered photos for fast-lane access).
- Face Retrieval: Finding matching faces in a database.
- Facelt® uses real-time imaging and RF tags to verify identities, improving border security and efficiency.

2. Human Movement and Expression Recognition:

- Systems can track movements (e.g., of heads, hands) or recognize expressions like smiles or anger (Pentland, 1997).
- This ties into emotion recognition (Picard, 1997) for better human-computer interaction.

3. Video Search with Named Faces:

- The Informedia Digital Video Library (Wactlar et al., 2000) extracts data from video/audio and supports full-content search.
- Its "named face" feature links names to faces, allowing searches like "find videos with this person."

Performance Metrics

- Systems like QBIC and Facelt® are evaluated using precision (how many retrieved items are relevant) and recall (how many relevant items were retrieved), metrics borrowed from text retrieval.
- **Challenges and Future Goals**
 - Object Identification: Fully automatic, domain-independent object recognition is still difficult, so manual or semi-automated annotation tools are used.
 - Semantic Access: The ultimate goal is to enable semantic-based retrieval (e.g., "find images of happy people at a beach") rather than just visual feature matching.

Video Retrieval

- **Broadcast News Navigator (BNN)**
- BNN is a web-based tool that automates the capture, annotation, segmentation, summarization, and visualization of broadcast news video. It integrates text, speech, and image processing to enable content-based search and retrieval, addressing the inefficiencies of manual video annotation.

Query Interface

Users can search across 30 news sources, select date ranges (e.g., Feb 27 to Mar 12, 2000), and search using keywords, closed captions, speech transcriptions, or named entities (people, locations, organizations).

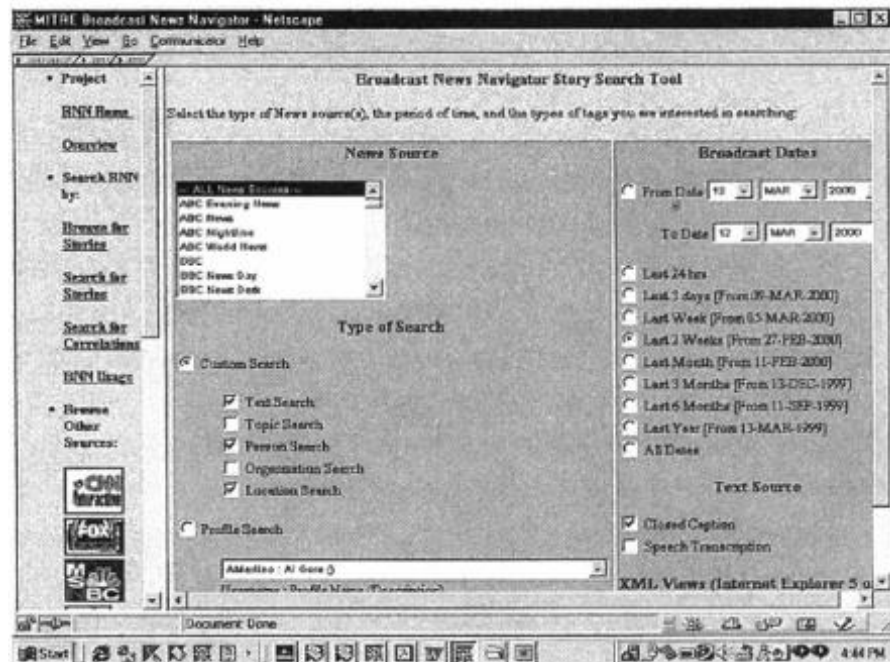


Figure 10.5a. Initial Query Page

Detailed Query

BNN generates a custom query page with menus of extracted entities (e.g., "George Bush," "New York") and keywords ("presidential primary") to refine searches.

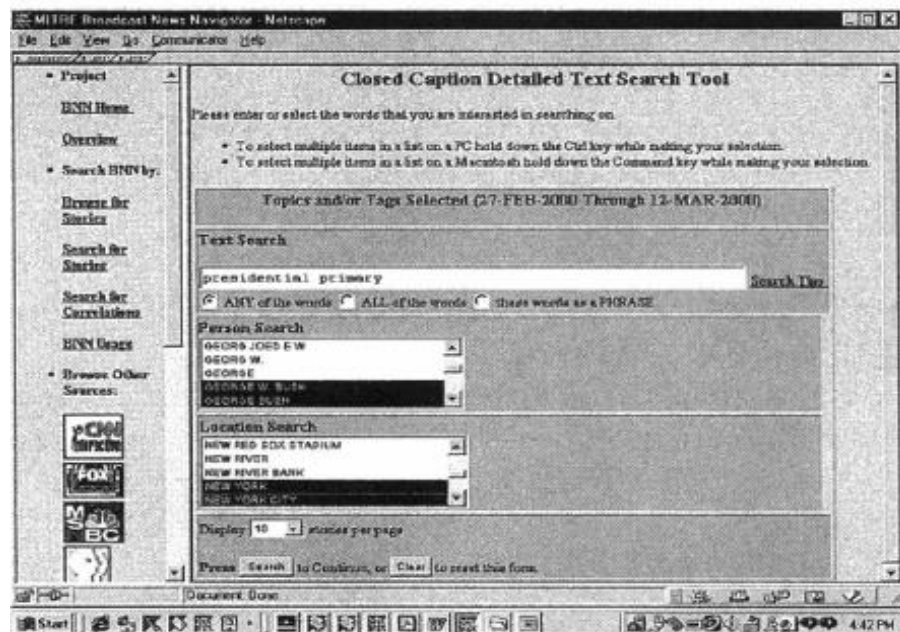


Figure 10.5b. Detailed Content-based Query

Figure 10.5. Broadcast News Navigator Query by Content



Figure 10.6b. "Al Gore" Stories

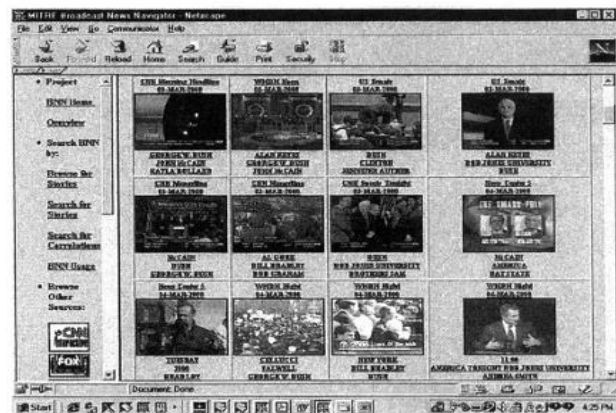


Figure 10.6a: "George Bush" Stories

BNN displays "story skims" with keyframes and the three most frequent named entities for each story. Fig. 10.6a shows stories about "George Bush," while 10.6b shows "Al Gore" stories.

Story Details

Selecting a story (e.g., a March 5 story about Al Gore) shows detailed closed captions, including delegate counts from the presidential primary.

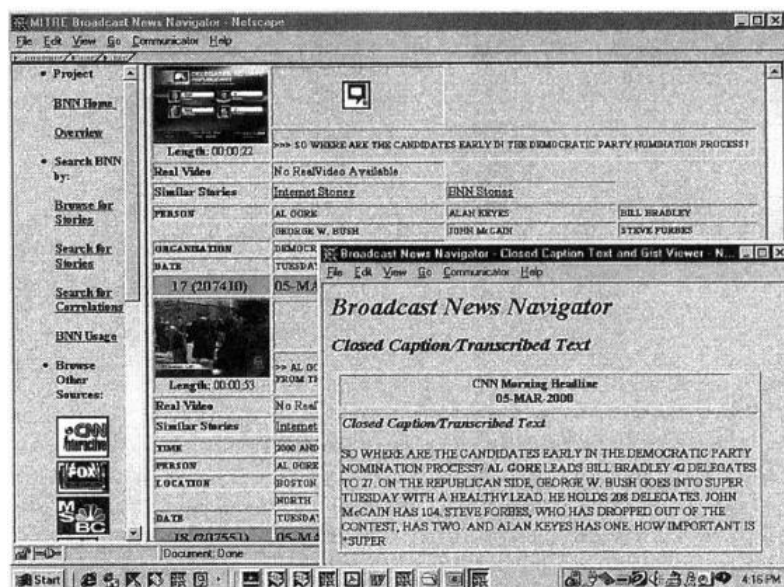


Figure 10.6c: BNN Story Detail Retrieval of "presidential primary" stories

Figure 10.6: BNN Retrieval of "presidential primary" stories

GeoNODE (Geospatial News on Demand Environment)

GeoNODE builds on BNN by adding geospatial and temporal context to news analysis, focusing on topic detection and tracking (TDT) for broadcast news and other sources.

Functionality

Timeline View (Fig. 10.7): GeoNODE displays a timeline of stories from sources like CNN and MSNBC, showing the frequency of named entities (e.g., "George W. Bush," "Al Gore") over time. Peaks indicate high coverage periods.

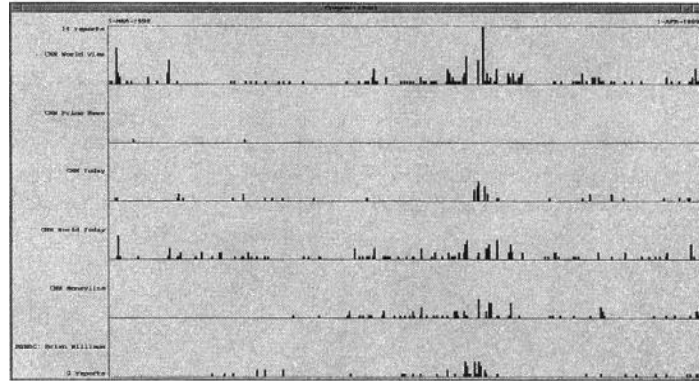


Figure 10.7: GeoNODE Time-line

Map Display

It visualizes the frequency of location mentions on a world map. Countries with more mentions are darker (e.g., North and South America are dark brown), and yellow circles indicate specific locations (larger circles for more mentions, e.g., South American capitals).

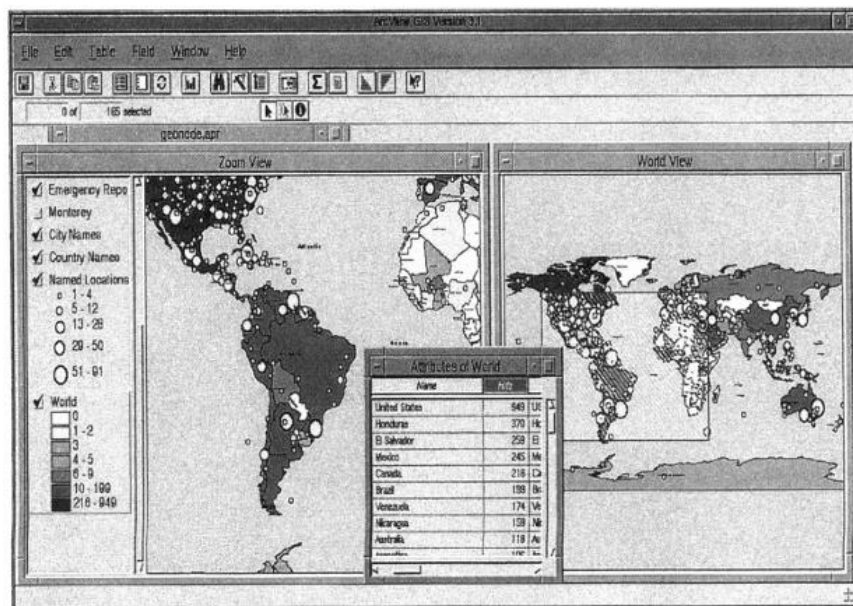


Figure 10.8: GeoNODE Map Display

- Performance:
 - In tests with 65,000 documents and 100 manually identified topics, GeoNODE identified over 80% of human-defined topics and detected 83% of stories within topics, with a 0.2% misclassification error—comparable to TDT initiative results.
- Applications:
 - GeoNODE helps analysts navigate news by topic, time, and place, using data from diverse sources like broadcast video and online newspapers.



TEXT SEARCH ALGORITHMS AND MULTIMEDIA INFORMATION RETRIEVAL

BNN and GeoNODE

- Both systems highlight advancements in multimedia analysis:
 - BNN focuses on story segmentation and content-based retrieval, improving speed and accuracy over manual methods.
 - GeoNODE extends this with topic detection, tracking, and geospatial visualization, enabling deeper analysis of news trends.
 - Future progress depends on multimedia corpora, standardized evaluation tasks, and machine learning to enhance extraction and analysis.